Paper 6 — Why Wide Deep Networks Can Reach Zero Loss

Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, Xiyu Zhai (ICML 2019)

Gradient Descent Finds Global Minima of Deep Neural Networks | RQ2: dynamics; RQ4: theory versus practice

Key: `du2019gradient` | **Tags:** A, D, G / F, B | **Tier:** Must read

The paper in one minute

Question: How can gradient descent reach *zero training loss* on a deep neural network even when the optimization problem is non-convex?
Model/Method: Sufficiently wide, randomly initialized deep networks and residual networks, analyzed through a data-dependent Gram matrix.
Finding: Under specified smoothness, data, width, and step-size conditions, full-batch GD reduces squared training loss *geometrically*. ResNets improve the *proved* dependence on depth from exponential to polynomial.
Why we care: Architecture and overparameterization can make a non-convex learning problem provably trainable without extra saddle-escape procedures.

1. Begin with what "global minimum" means here

We have n training samples (x_i, y_i) and neural-network predictions $f_\theta(x_i)$. Training minimizes a squared-error objective:

$$\mathcal{L}(\theta) = \frac{1}{2}\sum_{i=1}^{n}\big(f_\theta(x_i) - y_i\big)^2.$$

The loss is *non-convex in the weights*. Nevertheless, if all predictions match the labels, every squared term is zero and so $\mathcal{L} = 0$, the smallest possible loss. This is the **global training objective** of the theorem, *not* a claim about accuracy on new data.

2. Why width changes the way weights move

An extremely wide neural network can fit its training labels while moving each weight matrix only a little from random initialization. The authors exploit this stability: if the relevant prediction gradients barely change, their Gram matrix stays well conditioned, and training residuals contract.

Central intuition to retain

Rather than proving that *every* saddle or local minimum has a favorable shape, Du et al. analyze *the actual trajectory* of GD in a sufficiently overparameterized model. The key is that the local gradient relationships remain stable along that trajectory.

3. Three architecture-level results

Architecture	Width condition, schematic	Meaning
Plain deep network	$m \gtrsim \mathrm{poly}(n)\, 2^{O(H)}$	Linear-rate convergence, but costly dependence on depth H.
ResNet	$m \gtrsim \mathrm{poly}(n, H)$	Residual links improve the depth dependence.
Convolutional ResNet	$m \gtrsim \mathrm{poly}(n, H, p)$	Similar guarantee with patch count p.

Important precision. The displayed expressions suppress data-dependent eigenvalue and probability terms; these are sufficient *theoretical* widths, not recommendations for real models.

4. From weight updates to a prediction-space Gram matrix

Let $u(\theta) = (f_\theta(x_1), \ldots, f_\theta(x_n))^\top$ be all training predictions, and let $r(\theta) = u(\theta) - y$ collect their errors. Form the Jacobian and Gram matrix:

$$J(\theta) = \frac{\partial u(\theta)}{\partial \theta}, \qquad G(\theta) = J(\theta)J(\theta)^\top.$$

Read the shapes. J has one row for each training example and one column for each parameter. The entry G_{ij} measures alignment between the parameter-gradients of training predictions i and j. Since $G = JJ^\top$, it is positive semidefinite.

For a small GD step, linearizing the model predictions gives the *illustrative* update

$$r_{k+1} \approx \big(I - \eta G(\theta_k)\big) r_k.$$

A positive eigenvalue of G makes the corresponding residual component contract when η is chosen appropriately. A zero eigenvalue would leave one first-order error direction unchanged.

5. Worked example: see the geometric decrease numerically

Take a simple *fixed-Gram teaching model* with two residual coordinates. This is not a special case of all the assumptions in the deep-network theorem; it is an exact example of the *linearized* mechanism.

$$K = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{2} \end{bmatrix}, \qquad r_0 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \qquad \eta = 1.$$

Step 1. In this model, $r_{k+1} = (I - \eta K) r_k$, so

$$r_1 = \begin{bmatrix} 0 \\ -\frac{1}{2} \end{bmatrix}, \qquad r_2 = \begin{bmatrix} 0 \\ -\frac{1}{4} \end{bmatrix}, \qquad r_3 = \begin{bmatrix} 0 \\ -\frac{1}{8} \end{bmatrix}.$$

Step 2. Compute $\mathcal{L}_k = \frac{1}{2}\|r_k\|^2$:

Iteration	Residual r_k	Training loss $\mathcal{L}_k$
0	$(1, -1)$	1.00000
1	$(0, -0.5)$	0.12500
2	$(0, -0.25)$	0.03125
3	$(0, -0.125)$	0.00781

Step 3. The smallest eigenvalue here is $0.5 > 0$. Thus even the slowest residual direction shrinks. If it were 0, a component might never improve in this linearized system.

What makes the real paper harder

A neural network does *not* have a fixed K during training. The authors show the relevant Gram matrix starts with a positive lower eigenvalue and remains sufficiently close to its initial counterpart because the wide network's weights move only a limited distance.

6. How the proof converts Gram stability into convergence

The core argument has three linked stages:

1. **Good initial geometry.** Under nondegenerate training inputs and suitable smooth activations, the limiting data-dependent Gram matrix $K^{(H)}$ is positive definite. Define $\lambda_0 = \lambda_{\min}(K^{(H)}) > 0$.
2. **Small movement of weights.** Sufficient width and a suitably small full-batch GD step keep each trained weight matrix near its Gaussian random initialization.
3. **Persistent contraction.** This stability controls the changing Gram matrix and nonlinear remainder terms, so the training loss continues to shrink at a geometric rate.

Theorems 5.1 (plain network) and 6.1 (ResNet) give high-probability bounds of the form

$$\boxed{\mathcal{L}(\theta_k) \le \left(1 - \frac{\eta\lambda_0}{2}\right)^k \mathcal{L}(\theta_0).}$$

Meaning. As long as $0 < 1 - \eta\lambda_0/2 < 1$, the repeated multiplicative factor drives training loss toward zero. The allowable η is restricted by the theorem; this equation does not justify choosing arbitrarily large steps.

7. Why residual networks help: a short mathematical intuition

In an ordinary deep network, small perturbations can compound through successive layers. The proof for such networks incurs factors scaling exponentially in depth. A residual network instead passes an identity path alongside each learned transformation, with depth-dependent scaling.

$$x^{(h)} = x^{(h-1)} + \frac{c_{\text{res}}}{H\sqrt{m}}\,\sigma\left(W^{(h)}x^{(h-1)}\right).$$

Illustration, not an exact theorem constant. Compare multiplying a perturbation by 2 in every layer with multiplying by $1 + 1/H$ in every layer:

$$2^H \text{ grows exponentially,} \qquad \left(1 + \frac{1}{H}\right)^H \longrightarrow e.$$

For $H = 10$, the first factor is 1024, whereas the second is about 2.59. Residual links do not make the loss convex; rather, their structure makes perturbations more manageable in the analysis.

Result	What changes with depth H
Theorem 5.1: plain network	Sufficient width and step-size bounds may deteriorate exponentially in H.
Theorem 6.1: ResNet	Sufficient width and rate become polynomial in H (with other data-dependent factors).
Theorem 7.1: conv. ResNet	Similar high-probability convergence with dependence on patch count p.

Do not confuse: "Linear convergence" describes *geometric* reduction of optimization error. It does not mean a linear activation function, nor that the loss landscape is globally convex.

8. What this paper actually contributes

1. **Deep-network GD theory:** geometric convergence toward zero squared training loss at finite but sufficiently large width.
2. **Architecture-sensitive guarantee:** a polynomial rather than exponential *proved* depth dependence for scaled residual networks.
3. **ConvNet extension:** corresponding analysis for convolutional ResNets.
4. **Proof perspective:** stability of the sample-wise Gram matrix along the optimization trajectory, rather than a claim about all critical points in the full landscape.

9. Assumptions and limitations you must not lose

Important assumption	Boundary on the conclusion
Squared training loss, normalized and nondegenerate training inputs, bounded labels	Zero *training* loss does not imply good generalization.
Gaussian random initialization, sufficiently large width, carefully bounded full-batch GD learning rate	Not guaranteed for small networks, arbitrary initializations, or ordinary minibatch SGD.
Smooth Lipschitz activations satisfying the paper's additional analytic/nonpolynomial conditions for Gram positivity	Does not directly establish the theorem for nonsmooth ReLU.
Analyzed fully connected / properly scaled ResNet / convolutional ResNet architectures	Not a theorem for every deep architecture or all non-convex objectives.

The width bounds can be *very large*, and depend on λ_0 and failure probability δ: these are sufficient, not necessary, conditions.

10. The connection to our earlier papers

Paper	Distinction to remember
Saxe (2014)	Depth and initialization shape learning speed; Du gives rigorous zero-loss convergence in a special wide-network regime.
Dauphin (2014)	Curvature can hamper training; Du avoids explicit curvature interventions by assuming stable overparameterized dynamics.
Lee (2016)	Almost-sure strict-saddle avoidance is weaker than zero-loss convergence in the specific models studied by Du.
Jin (2017)	Perturbations yield finite-time near-second-order-stationarity generally; Du needs stronger structural conditions but reaches global *training* optimum.
Li (2018)	Architecture affects visualized geometry; Du provides complementary mathematical convergence theory.

11. Quick reference and what to read

Object	What it means
$r = u(\theta) - y$	Vector of residuals on the training data.
$J = \partial u / \partial \theta$	Jacobian: how training predictions depend on parameters.
$G = JJ^\top$	Gram matrix of prediction gradients.
$\lambda_0 > 0$	Positive lower bound enabling residual contraction.

Read carefully: Sections 4 and 6; Theorems 5.1 and 6.1. **Skim:** Section 7 and lengthy technical bounds after understanding their assumptions.

One-paragraph note for Papers Master

Du et al. (2019) prove geometric convergence to zero squared training loss for sufficiently wide deep networks via Gram-matrix stability. The analyzed ResNets have polynomial rather than exponential depth dependence in sufficient-width bounds. This requires smooth activations, nondegenerate data, Gaussian initialization, and small full-batch GD steps; the result says nothing about generalization.